

Assignment#3  
Muhammad Qasim Ali  
BCS-6A

Task1:

Code:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv)
{ int rank, size, number;

  MPI_Init(&argc, &argv);
  MPI_Comm_rank(MPI_COMM_WORLD,
    &rank);

  if (rank == 0) {
    number = 42; // Initialized
    value }

  MPI_Bcast(&number, 1, MPI_INT, 0, MPI_COMM_WORLD);

  printf("Process %d received number %d\n", rank, number);

  MPI_Finalize();
  return 0;
}
```

Output:



```
qasim-ali-khan@qasim-ali-khan-eve-intel-360s-5G:~/Ass3$ mpicc task1.c -o task1
qasim-ali-khan@qasim-ali-khan-eve-intel-360s-5G:~/Ass3$ mpirun -np 4 ./task1
Process 0 received number 42
Process 1 received number 42
Process 2 received number 42
```

**Q: If a non-root process changes the value before the broadcast?**

Ans:

The broadcast will overwrite any local changes with the value from the root (process 0).

- **Constraints when calling MPI\_Bcast?**  
All processes in the communicator must call MPI\_Bcast **with the same datatype, count, and root**, and participate in the call collectively.
- **Difference from point-to-point send/receive?**  
MPI\_Bcast is **collective** and involves **all processes**, making it more scalable for one-to-many communication. Point-to-point operations (MPI\_Send/MPI\_Recv) are explicit and pairwise.

Task2:

Code:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv)
{ int rank, size;
  int data[16], recv_data[4];

  MPI_Init(&argc, &argv);
  MPI_Comm_rank(MPI_COMM_WORLD,
    &rank); MPI_Comm_size(MPI_COMM_WORLD,
    &size);

  if (rank == 0) {
    for (int i = 0; i < 16; i++) {
      data[i] = i + 1;
    }
  }

  MPI_Scatter(data, 4, MPI_INT, recv_data, 4, MPI_INT, 0, MPI_COMM_WORLD);

  for (int i = 0; i < 4; i++) {
    recv_data[i] *= 2;
  }

  MPI_Gather(recv_data, 4, MPI_INT, data, 4, MPI_INT, 0, MPI_COMM_WORLD);

  if (rank == 0) {
    printf("Final array:
    ");
    for (int i = 0; i < 16; i++) {
      printf("%d ", data[i]);
    }
    printf("\n");
  }

  MPI_Finalize();
  return 0;
}
```

Output:

```
Process 2 received number 42
```

```
qasim-ali-khan@qasim-ali-khan-eve-intel-360s-5G :~/Ass3$ mpicc task2.c -o task2
```

```
qasim-ali-khan@qasim-ali-khan-eve-intel-360s-5G :~/Ass3$ mpirun -np 4 ./task2
```

```
Final array: 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32
```

**Q3:**

**Ans:**

**a. What if the number of processes is not evenly divisible by the data size?**

Ans: MPI\_Scatter and MPI\_Gather require equal counts per process. Uneven distribution would cause incorrect behavior or memory errors unless custom logic (e.g., MPI\_Scatterv) is used.

**b. Handle uneven distribution (optional):**

Ans: You'd need to use MPI\_Scatterv and MPI\_Gatherv with variable counts and displacements per process.

### Task3:

#### Code:

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(int argc, char** argv)
{
    int rank, size, number;
    int all_numbers[6];
    int max_value;
    float average, total;
    double t1, t2;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD,
    &rank); MPI_Comm_size(MPI_COMM_WORLD,
    &size);

    srand(time(NULL) + rank);
    number = rand() % 100 + 1;

    t1 = MPI_Wtime();
    MPI_Allgather(&number, 1, MPI_INT, all_numbers, 1, MPI_INT,
    MPI_COMM_WORLD); t2 = MPI_Wtime();

    if (rank == 0) {
        printf("All numbers:
        ");
        for (int i = 0; i < size; i++) {
            printf("%d ",
            all_numbers[i]);
        }
        printf("\nAllgather Time: %f seconds\n", t2 -
        t1); }

    t1 = MPI_Wtime();
    MPI_Reduce(&number, &max_value, 1, MPI_INT, MPI_MAX, 0,
    MPI_COMM_WORLD); t2 = MPI_Wtime();

    if (rank == 0) {
        printf("Maximum value: %d\n",
        max_value); printf("Reduce Time: %f
        seconds\n", t2 - t1);
    }

    MPI_Allreduce(&number, &total, 1, MPI_FLOAT, MPI_SUM,
    MPI_COMM_WORLD); average = total / size;

    printf("Process %d: Average = %.2f\n", rank, average)
```

```
MPI_Finalize();  
return 0;  
}
```

Output:

```
qasim-ali-khan@qasim-ali-khan-eve-intel-360s-5G  
qasim-ali-khan@qasim-ali-khan-eve-intel-360s-5G :~/Ass3$ mpicc task3.c -o task3  
jarrar-khan@jarrar-khan-HP-EliteBook-830-G5:~/Ass3$ mpirun -np 4 ./task3  
All numbers: 7 24 63 34  
Allgather Time: 0.000336 seconds  
Maximum value: 63  
Reduce Time: 0.000025 seconds  
Process 0: Average = 0.00  
Process 1: Average = 0.00  
Process 2: Average = 0.00  
Process 3: Average = 0.00
```

**Q4:**

**Ans:**

**a. Why is MPI\_Allgather more expensive than MPI\_Gather?**

Ans: MPI\_Allgather sends data **to all processes**, increasing the communication cost.  
MPI\_Gather only sends data to **one process (root)**.

**b. Can MPI\_Allreduce substitute Reduce + Broadcast?**

Ans: Yes. MPI\_Allreduce performs a **reduction + broadcast in one step**, making it more efficient and easier to use.

**c. Challenges with large data (e.g., 1 million elements)?**

Ans: Increased memory consumption and communication time. It may require optimizations like:

- Buffering,
- Compression,
- Overlapping communication with computation,
- Efficient use of derived datatypes.

